الهيئــــة الاتحـــادية للهويـــة
والجنسيـة والجمارك وأمن المنافـذ
FEDERAL AUTHORITY FOR IDENTITY,
CITIZENSHIP, CUSTOMS & PORT SECURITY

ID Card Toolkit Developer Guide .NET

Version 1.23

March 2026

CONFIDENTIAL

Federal Authority for Identity, Citizenship, Customs & Port Security (ICP)
United Arab Emirates

Table of Contents

Revision History

Version	Date	Description of Changes
1.0	23rd Feb, 2018	Release Version 1.0.0
1.1	30th Mar, 2018	Release Version 1.0.3
1.2	28th May, 2018	Release Version 1.0.5
1.3	12th Jun, 2018	Release Version 1.0.6
1.4	19th Sept, 2018	Release Version 1.0.8
1.5	31st Oct, 2018	Release Version 1.0.9; Included API to validate Toolkit Response
1.6	12th Dec, 2018	Release Version 1.0.13
1.7	16th Feb, 2019	Release Version 1.1.0
1.8	19th Mar, 2019	Release Version 1.2.0
1.9	03rd Apr, 2019	Release Version 1.3.0
1.10	13th May, 2019	Release Version 1.4.0
1.11	11th Jun, 2019	Release Version 1.4.1
1.12	17th Jun, 2019	Release Version 1.4.3
1.13	31st Jul, 2019	Release Version 1.4.3; Included APIs for Card Update functionality.
1.14	04th Mar, 2020	Release Version 1.4.5
1.15	01st Apr, 2020	Release Version 1.4.5-r2
1.16	05th Apr, 2020	Release Version 1.4.6
1.17	02nd Nov, 2020	Release Version 1.4.8
1.18	24th May, 2021	Release Version 1.4.9
1.19	04th Oct, 2021	Release Version 2.0.2
1.20	12th Apr, 2022	Release Version 2.0.3
1.21	13th Jun, 2022	Release Version 2.0.5
1.22	20th Dec, 2023	Release Version 3.0.0
1.23	Mar 2026	Toolkit v3.0.5. Added complete data model and enumeration reference. Updated environment setup for modern .NET. Improved code samples.

Abbreviations

Abbreviation	Description
CA	Certificate Authority
CAdES	CMS Advanced Electronic Signatures
CMS	Cryptographic Message Syntax
EF	Elementary File
ICP	Federal Authority for Identity, Citizenship, Customs & Port Security
IDE	Integrated Development Environment
MRZ	Machine Readable Zone
NFC	Near Field Communication
PAdES	PDF Advanced Electronic Signatures
PDF	Portable Document Format
SP	Service Provider
VG	Validation Gateway
WPF	Windows Presentation Foundation
XAdES	XML Advanced Electronic Signatures
XML	Extensible Markup Language

1. Introduction

The Federal Authority for Identity, Citizenship, Customs & Port Security (ICP) developed the ID Card Toolkit to address the requirements of service providers (SP) to integrate, into their business applications: Identification, Authentication, Digital Signature and Non-repudiation services, around the capabilities of the Emirates ID Card. The Toolkit is comprised of a number of Software Development Kits (SDK) supporting different programming languages and platforms.

1.1. Document Purpose

This document is a developer's guide to support the development of .NET applications that integrate the ICP ID Card Toolkit SDK. It covers environment setup, a quick-start tutorial, and the complete API reference for all Toolkit classes, methods, data models, and enumerations.

2. Development Environment Setup

The following environment setup is based on Visual Studio on Windows. The setup may vary slightly when using different versions of Visual Studio.

2.1. Prerequisites

- .NET Framework 4.6.2 or above
- ICP ID Card Toolkit .NET SDK
- IDE: Visual Studio 2019 or above

2.2. Environment Setup

1. Create a Visual C# WPF Application project in Visual Studio.
2. Set the minimum .NET Framework level to 4.6.2 or above.
3. Add `IDCardToolkit.dll` to the current working directory and then add it as a reference under the *References* node of the project.
4. Add the native libraries used by the Toolkit to the current working directory of the project.

> **Note:** There are different native libraries for the 32-bit and 64-bit processor architectures. These can be found in the `x86` and `x64` subdirectories of the SDK distribution. Ensure the correct architecture matches your application build target.

3. Developing Your First Program

The following example shows the sequence of API calls required to develop an application using the Toolkit .NET SDK. The example illustrates how to use the Read Public Data service. To use other services, follow a similar approach.

Refer to the Toolkit API Reference (Section 5) for details of all available services. A sample WPF program is also provided with the SDK.

3.1. Step 1: Initialize the Toolkit

Toolkit initialization is performed by the constructor of the `Toolkit` class which takes two parameters:

- `inProcessMode`: When `true`, the Toolkit runs in-process (inside the application process). When `false`, API requests are forwarded to the Toolkit Agent service. See the "ID Card Toolkit Technical Overview" document for details about integration modes.
- `configParams`: Application-specific configuration parameters as a string. Refer to the "ID Card Toolkit Programmer's Reference" for the full list.

```
// Toolkit Namespace
using AE.EmiratesId.IdCard;
using AE.EmiratesId.IdCard.DataModels;

try
{
    // Create an object and initialize Toolkit in in-process mode
    Toolkit toolkitObj = new Toolkit(true, configParams);
}
catch (ToolkitException tEx)
{
    Console.WriteLine($"Toolkit error {tEx.Code}: {tEx.Message}");
}
```

3.2. Step 2: List Connected Readers

Retrieve a list of connected smartcard readers using the `ListReaders` method. This returns an array of `CardReader` objects. If no compatible readers are found, an empty array is returned.

```
try
{
    CardReader[] readers = toolkitObj.ListReaders();
    Console.WriteLine($"Found {readers.Length} reader(s)");
}
catch (ToolkitException tEx)
{
    Console.WriteLine($"Error: {tEx.Message}");
}
```

3.3. Step 3: Connect to a Card Reader

Connect to a card reader by calling the `Connect` method on a `CardReader` object returned by `ListReaders`.

```
try
{
    CardReader cardReader = readers[0];
    cardReader.Connect();
}
catch (ToolkitException tEx)
{
    Console.WriteLine($"Connect failed: {tEx.Message}");
}
```

> **Note:** To directly access a card reader with an Emirates ID card inserted, use the `GetReaderWithEmiratesId` method. This avoids the need to call `ListReaders` and iterate over the results.
>
> ```
> CardReader cardReader = toolkitObj.GetReaderWithEmiratesId();
> cardReader.Connect();
> ```

3.4. Step 4: Read Public Data

Call `ReadPublicData` to retrieve public data from the connected card. The application must generate a Request ID before each service call. Refer to the "ID Card Toolkit Programmer's Reference" for guidelines on generating the Request ID.

```
try
{
    CardPublicData publicData = cardReader.ReadPublicData(
        requestId,
        readNonModifiableData: true,
        readModifiableData: true,
        readPhotography: true,
        readSignatureImage: false,
        readAddress: false);

    // Access the card holder ID number
    string idNumber = publicData.IdNumber;

    // Access the card holder photo (JPEG bytes)
    byte[] photo = publicData.CardHolderPhoto;

    // Access the digitally signed Toolkit Response XML
    string xmlResponse = publicData.XmlString;
}
catch (ToolkitException tEx)
{
    Console.WriteLine($"ReadPublicData failed: {tEx.Message}");
}
```

If successful, the method returns a `CardPublicData` object containing the requested data. The `XmlString` property contains the digitally signed Toolkit Response XML. Applications should validate the response digital signature and Request ID to verify authenticity. Refer to the sample program provided with the SDK.

3.5. Step 5: Disconnect

Disconnect from the card when done. The card can be safely removed after disconnect.

```
try
{
    cardReader.Disconnect();
}
catch (ToolkitException tEx)
{
    Console.WriteLine($"Disconnect failed: {tEx.Message}");
}
```

3.6. Step 6: Cleanup

Release all Toolkit resources by calling `Cleanup` when the application no longer needs the Toolkit.

```
toolkitObj.Cleanup();
```

4. Toolkit API Reference

4.1. Namespace

The `AE.EmiratesId.IdCard` namespace contains the `Toolkit` and `CardReader` classes. The `AE.EmiratesId.IdCard.DataModels` namespace contains the data model classes.

Item	Description
Assembly	`IDCardToolkit.dll` – add to the References of the application project
Namespace	`AE.EmiratesId.IdCard` – Toolkit classes and methods
Data Models	`AE.EmiratesId.IdCard.DataModels` – response and data model classes

Class	Description
`Toolkit`	Provides methods to initialize the Toolkit and manage connected smartcard readers.
`CardReader`	Provides all methods to perform operations with the Emirates ID Card.
`SignatureValidator`	Provides methods to validate Toolkit XML Response signatures.

4.2. Class: Toolkit

This class provides methods required to initialize the Toolkit and get the connected smartcard readers.

4.2.1. Constructor: Toolkit

Initializes the Toolkit context and prepares the SDK for use.

Syntax

```
public Toolkit(bool inProcessMode, string configParams)
```

Parameter	Type	Description
`inProcessMode`	`bool`	When `true`, the Toolkit runs in-process. When `false`, API requests are forwarded to the Toolkit Agent.
`configParams`	`string`	Configuration parameters as a string. If a file is used, read its content and provide it as a string. Pass `null` to use the default `config_ap` file from the Toolkit directory. Refer to the Programmer's Reference for details.

Return: In case of error, the constructor throws `ToolkitException`.

4.2.2. Method: GetToolkitVersion

Returns the version of the Toolkit being used by the application.

Syntax

```
public string GetToolkitVersion()
```

Parameters: None

Return: `string` – Toolkit version in the format `major.minor.patch`. Throws `ToolkitException` on error.

4.2.3. Method: Cleanup

Releases resources used by the Toolkit and closes the service context. In in-process mode, cleanup is performed within the Toolkit library. In agent mode, a cleanup request is sent to the agent.

Syntax

```
public void Cleanup()
```

Parameters: None

Return: `void`. Throws `ToolkitException` on error.

4.2.4. Method: ListReaders

Lists all compatible smartcard readers connected to the system.

Syntax

```
public CardReader[] ListReaders()
```

Parameters: None

Return: `CardReader[]` – array of connected smartcard readers. Returns an empty array if no readers are found. Throws `ToolkitException` on error.

4.2.5. Method: GetReaderWithEmiratesId

Checks all smartcard readers and returns the first reader with an Emirates ID Card inserted.

Syntax

```
public CardReader GetReaderWithEmiratesId()
```

Parameters: None

Return: `CardReader` – the reader with an Emirates ID Card inserted. Throws `ToolkitException` on error.

4.2.6. Method: PrepareRequest

Establishes a secure context for execution of a Toolkit service. The Request Handle returned in the Toolkit Response XML is used by applications to encode sensitive information (User ID, Password, PIN) to protect against sniffing and replay attacks.

Syntax

```
public string PrepareRequest(string requestId)
```

Parameter	Type	Description
`requestId`	`string`	Application-generated request identifier. Refer to the Programmer's Reference for guidelines on generating the Request ID.

Return: `string` – 8-byte request handle in Base64 format. Throws `ToolkitException` on error.

4.2.7. Method: RegisterDevice

Registers a device with the Validation Gateway (VG) against the Service Provider (SP) license. `PrepareRequest` must be called successfully before this method. The Request Handle is required to encode the `encodedUserId` and `encodedPassword` parameters. Only registered devices can access Toolkit services (except offline public data reading).

Syntax

```
public RegisterDeviceResponse RegisterDevice(
    string encodedUserId,
    string encodedPassword,
    string deviceReferenceId)
```

Parameter	Type	Description
`encodedUserId`	`string`	SP User ID, encrypted and Base64-encoded per the Programmer's Reference.
`encodedPassword`	`string`	Password for the User ID, encrypted and Base64-encoded per the Programmer's Reference.
`deviceReferenceId`	`string`	Unique device ID generated by the SP for internal device management. ICP stores this along with device information during registration.

Return: `RegisterDeviceResponse` – contains the Device Registration Identifier. The `XmlString` property returns the XML response for signature validation. Throws `ToolkitException` on error.

4.2.8. Method: GetDeviceId

Retrieves the unique device identifier. This identifier is required to deregister a device through the ICP Web Service REST API.

Syntax

```
public string GetDeviceId()
```

Parameters: None

Return: `string` – the device identifier. Throws `ToolkitException` on error.

4.2.9. Method: GetDataProtectionKey

Retrieves the VG public key used for data protection. This key is used to encode sensitive parameters (User ID, Password, PIN) required by Toolkit services.

Syntax

```
public DataProtectionKey GetDataProtectionKey()
```

Parameters: None

Return: `DataProtectionKey` – contains the data protection public key. The `PublicKey`, `Modulus`, and `Exponent` properties provide the key components. Refer to the Programmer's Reference for encoding procedures. Throws `ToolkitException` on error.

4.2.10. Method: ParseMRZData

Parses a Machine Readable Zone (MRZ) string and retrieves the corresponding attributes.

Syntax

```
public MRZData ParseMRZData(string mrz)
```

Parameter	Type	Description
`mrz`	`string`	The MRZ string to parse.

Return: `MRZData` – parsed MRZ attributes accessible through the structure properties. Throws `ToolkitException` on error.

4.2.11. Method: GetLicenseExpiryDate

Returns the expiry date of the Toolkit SDK license issued to the Service Provider.

Syntax

```
public string GetLicenseExpiryDate()
```

Parameters: None

Return: `string` – the license expiry date. Throws `ToolkitException` on error.

4.2.12. Method: GetConfigCertificateExpiryDate

Returns the expiry dates of certificates corresponding to the Toolkit SDK configuration files.

Syntax

```
public ConfigCertExpiryDate GetConfigCertificateExpiryDate()
```

Parameters: None

Return: `ConfigCertExpiryDate` – certificate expiry dates for each configuration file. Throws `ToolkitException` on error.

4.3. Class: CardReader

This class provides all methods required to perform operations with the Emirates ID Card after a successful `Connect` operation.

4.3.1. Properties

Property	Type	Description
`Name`	`string`	The name of the smartcard reader.
`ReaderSerialNumber`	`string`	The serial number of the smartcard reader.

4.3.2. Method: Connect

Establishes a connection to the ID card in the reader. This method must be called each time an ID card is presented or when the reader is reconnected. Once connected, multiple Toolkit services can be invoked. Reusing the connection handle for multiple services within a single ID card session improves performance.

Syntax

```
public void Connect()
```

Parameters: None

Return: `void`. Throws `ToolkitException` on error.

4.3.3. Method: Disconnect

Disconnects from the connected reader. The Emirates ID Card can be safely removed after disconnect.

Syntax

```
public void Disconnect()
```

Parameters: None

Return: `void`. Throws `ToolkitException` on error.

4.3.4. Method: GetCardVersion

Retrieves the version number of the connected Emirates ID Card.

Syntax

```
public int GetCardVersion()
```

Parameters: None

Return: `int` – the card version number. Throws `ToolkitException` on error.

Value	Card Version
2	V2
3	V3
4	V4
5	V5

V5 is the latest generation card with enhanced security. Full read, PKI, and biometric operations are supported.

4.3.5. Method: GetInterfaceType

Returns the card communication interface type. Emirates ID Cards can be accessed through contact or contactless (NFC) interface.

Syntax

```
public int GetInterfaceType()
```

Parameters: None

Return: `int` – the interface type. Throws `ToolkitException` on error.

Value	Interface
1	Contact
2	NFC

4.3.6. Method: SetNfcAuthenticationParameters (MRZ)

Sets the NFC authentication parameters using the MRZ string from the back of the ID card. How to acquire the MRZ string is outside the scope of the Toolkit (e.g., OCR library or MRZ scanner).

Syntax

```
public void SetNfcAuthenticationParameters(string mrzData)
```

Parameter	Type	Description
`mrzData`	`string`	Scanned Machine Readable Zone (MRZ) data from the back of the ID Card.

Return: `void`. Throws `ToolkitException` on error.

4.3.7. Method: SetNfcAuthenticationParameters (Card Attributes)

Sets the NFC authentication parameters using specific card attributes printed on the card surface.

Syntax

```
public void SetNfcAuthenticationParameters(
    string cardNumber,
    string dateOfBirth,
    string expiryDate)
```

Parameter	Type	Description
`cardNumber`	`string`	Card number of the Emirates ID Card holder.
`dateOfBirth`	`string`	Date of birth in `YYMMDD` format. Example: 12th April 1956 = `560412`.
`expiryDate`	`string`	Card expiry date in `YYMMDD` format. Example: 16th November 2027 = `271116`.

Return: `void`. Throws `ToolkitException` on error.

4.3.8. Method: CheckCardStatus

Validates the Emirates ID Card with the ICP Validation Gateway (VG) and returns the card status.

Syntax

```
public ToolkitResponse CheckCardStatus(string requestId)
```

Parameter	Type	Description
`requestId`	`string`	Application-generated request identifier.

Return: `ToolkitResponse` – contains the card status. The `Status` property returns the card status value. The `XmlString` property returns the XML response for signature validation. Throws `ToolkitException` on error.

4.3.9. Method: GetCSN

Retrieves the serial number of the connected Emirates ID Card.

Syntax

```
public string GetCSN()
```

Parameters: None

Return: `string` – the card serial number. Throws `ToolkitException` on error.

4.3.10. Method: ReadPublicData

Retrieves data stored on the public areas of the Emirates ID Card. Applications can use flags to select which data categories are returned, improving performance when only specific data is needed. The data is digitally signed for integrity and verified within the Toolkit.

Syntax

```
public CardPublicData ReadPublicData(
    string requestId,
    bool readNonModifiableData,
    bool readModifiableData,
    bool readPhotography,
    bool readSignatureImage,
    bool readAddress)
```

Parameter	Type	Description
`requestId`	`string`	Application-generated request identifier.
`readNonModifiableData`	`bool`	`true` to read non-modifiable data (name, nationality, etc.).
`readModifiableData`	`bool`	`true` to read modifiable data (occupation, employer, etc.).
`readPhotography`	`bool`	`true` to read the card holder photograph.
`readSignatureImage`	`bool`	`true` to read the handwritten signature image.
`readAddress`	`bool`	`true` to read home and work address data.

Return: `CardPublicData` – contains the requested public data. Key properties include `IdNumber`, `CardNumber`, `CardHolderPhoto`, `HolderSignatureImage`, `NonModifiablePublicData`, `ModifiablePublicData`, `HomeAddress`, and `WorkAddress`. The `XmlString` property returns the digitally signed XML response. Throws `ToolkitException` on error.

4.3.11. Method: ReadPublicDataEF

Retrieves Elementary File data from the public areas of the ID card. The digital signature of the EF data is verified when the `validateSignature` flag is set.

Syntax

```
public byte[] ReadPublicDataEF(
    PublicDataEFType publicDataEFType,
    bool validateSignature)
```

Parameter	Type	Description
`publicDataEFType`	`PublicDataEFType`	Enumeration value identifying the Elementary File to read. See the `PublicDataEFType` enumeration.
`validateSignature`	`bool`	`true` to validate the digital signature of the EF data.

Return: `byte[]` – the Elementary File data as a byte array. Throws `ToolkitException` on error.

4.3.12. Method: ParseEFData

Parses Elementary File data from the public areas of the ID card into a human-readable string.

Syntax

```
public string ParseEFData(byte[] ef_data)
```

Parameter	Type	Description
`ef_data`	`byte[]`	Elementary File data bytes to parse (obtained from `ReadPublicDataEF`).

Return: `string` – the parsed Elementary File data. Throws `ToolkitException` on error.

4.3.13. Method: PrepareRequest (CardReader)

Establishes a secure context for execution of a Toolkit service on the connected card. Functions identically to the `Toolkit.PrepareRequest` method but operates within the card reader context.

Syntax

```
public string PrepareRequest(string requestId)
```

Parameter	Type	Description
`requestId`	`string`	Application-generated request identifier.

Return: `string` – 8-byte request handle in Base64 format. Throws `ToolkitException` on error.

4.3.14. Method: GetPkiCertificates

Reads the authentication and signing certificates from the Emirates ID Card. `PrepareRequest` must be called successfully before this method.

Syntax

```
public CardCertificates GetPkiCertificates(string encodedPin)
```

Parameter	Type	Description
`encodedPin`	`string`	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: `CardCertificates` – contains `AuthenticationCertificate` and `SigningCertificate` as byte arrays. The `XmlString` property returns the XML response. Throws `ToolkitException` on error.

Exception Type	Meaning
`CardPinError`	Wrong PIN provided. `AttemptsLeft` indicates remaining attempts before the card is blocked.
`ToolkitError`	Toolkit-related error.

4.3.15. Method: AuthenticatePki

Authenticates the ID Card PIN and verifies the revocation status of the authentication certificate. `PrepareRequest` must be called successfully before this method.

Syntax

```
public ToolkitResponse AuthenticatePki(string encodedPin)
```

Parameter	Type	Description
encodedPin	string	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: `ToolkitResponse` – contains the PKI authentication status. Throws `ToolkitException` on error.

Exception Type	Meaning
CardPinError	Wrong PIN provided. `AttemptsLeft` indicates remaining attempts before the card is blocked.
ToolkitError	Toolkit-related error.

4.3.16. Method: SignData

Digitally signs plain data or a hash using the signature certificate of the ID card. `PrepareRequest` must be called successfully before this method.

Syntax

```
public SignatureResponse SignData(
    byte[] input,
    bool isInputHash,
    string encodedPin)
```

Parameter	Type	Description
input	byte[]	Data or hash to digitally sign.
isInputHash	bool	`true` if hash data is provided; `false` if plain data is provided.
encodedPin	string	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: `SignatureResponse` – contains the `Signature` as a byte array and the signing status. Throws `ToolkitException` on error.

Exception Type	Meaning
CardPinError	Wrong PIN provided. `AttemptsLeft` indicates remaining attempts before the card is blocked.
ToolkitError	Toolkit-related error.

4.3.17. Method: SignChallenge

Digitally signs challenge data using the authentication certificate of the ID card. Used to authenticate users through challenge signing and verification. `PrepareRequest` must be called successfully before this method.

Syntax

```
public SignatureResponse SignChallenge(
    byte[] input,
    bool isInputHash,
    string encodedPin)
```

Parameter	Type	Description
`input`	`byte[]`	Challenge bytes to digitally sign.
`isInputHash`	`bool`	`true` if hash data is provided; `false` if plain challenge data is provided.
`encodedPin`	`string`	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: `SignatureResponse` – contains the `Signature` as a byte array and the signing status. Throws `ToolkitException` on error.

Exception Type	Meaning
`CardPinError`	Wrong PIN provided. `AttemptsLeft` indicates remaining attempts before the card is blocked.
`ToolkitError`	Toolkit-related error.

4.3.18. Method: VerifySignature

Verifies a digital signature with the corresponding certificate provided by the application.

Syntax

```
public void VerifySignature(
    byte[] input,
    bool isInputHash,
    byte[] signature,
    byte[] certificate)
```

Parameter	Type	Description
`input`	`byte[]`	Data or hash corresponding to the digital signature.
`isInputHash`	`bool`	`true` if hash data is provided; `false` if plain data is provided.
`signature`	`byte[]`	The digital signature to verify.
`certificate`	`byte[]`	Certificate data for signature verification.

Return: `void`. Throws `ToolkitException` if verification fails or on error.

4.3.19. Method: GetFingerData

Returns finger index values and biometric reference identifiers of the fingerprints stored on the Emirates ID Card.

Syntax

```
public FingerData[] GetFingerData()
```

Parameters: None

Return: `FingerData[]` – array of finger data objects containing `FingerId` and `FingerIndex` values. Throws `ToolkitException` on error.

4.3.20. Method: AuthenticateBiometricOnServer

Validates the Emirates ID Card and performs biometric authentication with the VG service against a captured finger image.

Syntax

```
public ToolkitResponse AuthenticateBiometricOnServer(
    string requestId,
    FingerIndex fingerIndex,
    int sensorTimeout)
```

Parameter	Type	Description
`requestId`	`string`	Application-generated request identifier.
`fingerIndex`	`FingerIndex`	Finger index from a previous `GetFingerData` call. See the `FingerIndex` enumeration.
`sensorTimeout`	`int`	Timeout in seconds for the fingerprint sensor to capture an image.

Return: `ToolkitResponse` – contains the biometric authentication status. Throws `ToolkitException` on error.

4.3.21. Method: AuthenticateCardAndBiometric

Validates the Emirates ID Card and performs biometric authentication with the VG service against a captured finger image. Combines card validation and biometric authentication in a single operation.

Syntax

```
public ToolkitResponse AuthenticateCardAndBiometric(
    string requestId,
    FingerIndex fingerIndex,
    int sensorTimeout)
```

Parameter	Type	Description
`requestId`	`string`	Application-generated request identifier.
`fingerIndex`	`FingerIndex`	Finger index from a previous `GetFingerData` call.
`sensorTimeout`	`int`	Timeout in seconds for the fingerprint sensor to capture an image.

Return: `ToolkitResponse` – contains the biometric authentication status. Throws `ToolkitException` on error.

4.3.22. Method: ResetPin

Resets the ID Card PIN using the VG service. Involves biometric authentication of the card holder. `PrepareRequest` must be called successfully before this method.

Syntax

```
public ToolkitResponse ResetPin(
    string encodedPin,
    FingerData fingerData,
    int sensorTimeout)
```

Parameter	Type	Description
`encodedPin`	`string`	New PIN, encrypted and Base64-encoded per the Programmer's Reference.
`fingerData`	`FingerData`	Finger data object from a previous `GetFingerData` call.
`sensorTimeout`	`int`	Timeout in seconds for the fingerprint sensor to capture an image.

Return: `ToolkitResponse` – contains the reset status in `ResponseStatus`. Throws `ToolkitException` on error.

Exception Type	Meaning
`CardPinError`	PIN-related error. `AttemptsLeft` indicates remaining attempts.
`ToolkitError`	Toolkit-related error.

4.3.23. Method: ResetPinWithoutAuthenticateBiometric

Resets the ID Card PIN using the VG service without biometric authentication. `PrepareRequest` must be called successfully before this method.

Syntax

```
public ToolkitResponse ResetPinWithoutAuthenticateBiometric(
    string encodedPin)
```

Parameter	Type	Description
`encodedPin`	`string`	New PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: `ToolkitResponse` – contains the reset status. Throws `ToolkitException` on error.

Exception Type	Meaning
`CardPinError`	PIN-related error. `AttemptsLeft` indicates remaining attempts.
`ToolkitError`	Toolkit-related error.

4.3.24. Method: UnblockPin

Unblocks the ID Card PIN after the card has been blocked due to multiple unsuccessful PIN attempts. Involves biometric authentication. `PrepareRequest` must be called successfully before this method.

Syntax

```
public ToolkitResponse UnblockPin(
    string encodedPin,
    FingerData fingerData,
    int sensorTimeout)
```

Parameter	Type	Description
`encodedPin`	`string`	New PIN, encrypted and Base64-encoded per the Programmer's Reference.
`fingerData`	`FingerData`	Finger data object from a previous `GetFingerData` call.
`sensorTimeout`	`int`	Timeout in seconds for the fingerprint sensor to capture an image.

Return: `ToolkitResponse` – contains the unblock status. Throws `ToolkitException` on error.

Exception Type	Meaning
`CardPinError`	PIN-related error. `AttemptsLeft` indicates remaining attempts.
`ToolkitError`	Toolkit-related error.

4.3.25. Method: UpdateData

Updates data on the Emirates ID Card. The update can be for existing data (modifiable data, address, family book) or for new containers that are personalized before the data is updated.

Syntax

```
public ToolkitResponse UpdateData(
    ContainerType fileType,
    string requestId)
```

Parameter	Type	Description
`fileType`	`ContainerType`	Identifier of the container to update. See the `ContainerType` enumeration.
`requestId`	`string`	Application-generated request identifier.

Return: `ToolkitResponse` – contains the update status. Throws `ToolkitException` on error.

4.3.26. Method: ReadData

Reads updated container data from the Emirates ID Card. The data is digitally signed for integrity and verified within the Toolkit.

Syntax

```
public DataContainer ReadData(
    FileType fileType,
    string requestId)
```

Parameter	Type	Description
`fileType`	`FileType`	Identifier of the container file to read. See the `FileType` enumeration.
`requestId`	`string`	Application-generated request identifier.

Return: `DataContainer` – contains the requested container data. Throws `ToolkitException` on error.

4.3.27. Method: ReadFamilyBookData

Reads the family book data from the Emirates ID Card. Family book data is a protected object requiring PIN authentication. `PrepareRequest` must be called successfully before this method.

Syntax

```
public CardFamilyBookData ReadFamilyBookData(string encodedPin)
```

Parameter	Type	Description
`encodedPin`	`string`	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: `CardFamilyBookData` – contains the family book data fields including `HeadOfFamily`, `Wives`, and `Children` collections. The `XmlString` property returns the XML response. Throws `ToolkitException` on error.

Exception Type	Meaning
`CardPinError`	Wrong PIN provided. `AttemptsLeft` indicates remaining attempts.
`ToolkitError`	Toolkit-related error.

4.4. Class: SignatureValidator

This class provides a method to validate Toolkit XML Response received from various API operations.

4.4.1. Constructor: SignatureValidator

Sets the attributes required to verify the Toolkit XML Response with the corresponding certificate and its chain.

Syntax

```
public SignatureValidator(
    byte[] certificateData,
    byte[] certificateChain)
```

Parameter	Type	Description
`certificateData`	`byte[]`	Certificate data for response verification.
`certificateChain`	`byte[]`	Issuer certificate chain data. Pass `null` to skip chain validation.

Return: In case of error, the constructor throws `ToolkitException`.

4.4.2. Method: ValidateToolkitResponse

Validates a Toolkit XML response signature.

Syntax

```
public ToolkitResponseData ValidateToolkitResponse(
    string toolkitResponse)
```

Parameter	Type	Description
`toolkitResponse`	`string`	Toolkit Response XML string (obtained from the `XmlString` property of response objects).

Return: `ToolkitResponseData` – contains the verified response attributes. Throws `ToolkitException` on error.

5. Advanced Digital Signature API Reference

The Toolkit provides APIs for creating and verifying advanced electronic signatures conforming to PAdES, XAdES, and CAdES standards. These APIs support multiple signature levels:

Signature Level	Description
Baseline-B	Basic signature with signer's certificate.
Baseline-T	Adds a trusted timestamp to the signature.
Baseline-LT	Adds certificate revocation data for long-term validation.
Baseline-LTA	Adds archival timestamp for long-term archival validation.

5.1. Class: CardReader (Digital Signature Methods)

5.1.1. Method: PadesSign

Digitally signs a PDF document in conformance with PAdES (PDF Advanced Electronic Signatures) per ETSI EN 319 142. `PrepareRequest` must be called successfully before this method.

Syntax

```
public ToolkitResponse PadesSign(
    SigningContext signingContext,
    string pdfFilePath,
    string signedPdfFilePath,
    PadesSignParams padesParams)
```

Parameter	Type	Description
`signingContext`	`SigningContext`	Signature properties including level, packaging mode, PIN, and optional TSA/OCSP URLs. See the `SigningContext` structure.
`pdfFilePath`	`string`	Path of the input PDF document.
`signedPdfFilePath`	`string`	Path to store the signed PDF document.
`padesParams`	`PadesSignParams`	PAdES-specific signature properties (visible signature, position, appearance). See the `PadesSignParams` structure.

Return: `ToolkitResponse` – contains the signing status. Throws `ToolkitException` on error.

Exception Type	Meaning
`CardPinError`	Wrong PIN provided. `AttemptsLeft` indicates remaining attempts.
`ToolkitError`	Toolkit-related error.

5.1.2. Method: PadesVerify

Verifies a PAdES-signed PDF document.

Syntax

```
public string PadesVerify(
    VerificationContext verificationContext,
    string signedPdfFilePath)
```

Parameter	Type	Description
`verificationContext`	`VerificationContext`	Verification context properties. See the `VerificationContext` structure.
`signedPdfFilePath`	`string`	Path of the signed PDF document to verify.

Return: `string` – verification report in XML format. Throws `ToolkitException` on error.

5.1.3. Method: XadesSign

Digitally signs an XML document in conformance with XAdES (XML Advanced Electronic Signatures) per ETSI EN 319 132. `PrepareRequest` must be called successfully before this method.

Syntax

```
public ToolkitResponse XadesSign(
    SigningContext signingContext,
    string xmlFilePath,
    string signedXmlFilePath)
```

Parameter	Type	Description
`signingContext`	`SigningContext`	Signature properties including level, packaging mode, and PIN.
`xmlFilePath`	`string`	Path of the input XML document.
`signedXmlFilePath`	`string`	Path to store the signed document. Pass `null` for detached mode.

Return: `ToolkitResponse` – in detached mode, the `DetachedSignature` property contains the signature. In enveloped/enveloping mode, the signature is embedded in the output file. Throws `ToolkitException` on error.

Signing Mode	signedXmlFilePath	DetachedSignature
Detached	`null`	Contains the signature
Enveloped / Enveloping	Path to signed document	`null`

Exception Type	Meaning
`CardPinError`	Wrong PIN provided. `AttemptsLeft` indicates remaining attempts.
`ToolkitError`	Toolkit-related error.

5.1.4. Method: XadesVerify

Verifies an XAdES-signed XML document.

Syntax

```
public string XadesVerify(
    VerificationContext verificationContext,
    string signedXmlFilePath,
    byte[] signature)
```

Parameter	Type	Description
`verificationContext`	`VerificationContext`	Verification context properties.
`signedXmlFilePath`	`string`	Path of the XML document to verify.
`signature`	`byte[]`	Detached signature bytes (for detached mode). Pass `null` for enveloped/enveloping.

Signing Mode	signedXmlFilePath	signature
Detached	Path to original XML	Detached signature bytes
Enveloped / Enveloping	Path to signed XML	`null`

Return: `string` – verification report in XML format. Throws `ToolkitException` on error.

5.1.5. Method: CadesSign

Digitally signs data in conformance with CAdES (CMS Advanced Electronic Signatures) per ETSI EN 319 122. `PrepareRequest` must be called successfully before this method.

Syntax

```
public ToolkitResponse CadesSign(
    SigningContext signingContext,
    string inputFilePath)
```

Parameter	Type	Description
`signingContext`	`SigningContext`	Signature properties. The `PackagingMode` must be set to `Detached`.
`inputFilePath`	`string`	Path of the input file to sign.

> **Note:** CAdES signing works only in detached mode.

Return: `ToolkitResponse` – the `DetachedSignature` property contains the CAdES signature. Throws `ToolkitException` on error.

Exception Type	Meaning
`CardPinError`	Wrong PIN provided. `AttemptsLeft` indicates remaining attempts.
`ToolkitError`	Toolkit-related error.

5.1.6. Method: CadesVerify

Verifies a CAdES signature.

Syntax

```
public string CadesVerify(
    VerificationContext verificationContext,
    string inputFilePath,
    byte[] signature)
```

Parameter	Type	Description
`verificationContext`	`VerificationContext`	Verification context properties.
`inputFilePath`	`string`	Path of the original document.
`signature`	`byte[]`	CAdES signature bytes to verify.

Return: `string` – verification report in XML format. Throws `ToolkitException` on error.

6. Data Models and Structures

6.1. Response Classes

6.1.1. ToolkitResponse

Base class for Toolkit service responses.

Property	Type	Description
`Service`	`string`	Name of the Toolkit service.
`Action`	`string`	Action performed.
`RequestId`	`string`	The request ID provided by the application.
`Nonce`	`string`	Server-generated nonce value.
`CorrelationId`	`string`	Correlation identifier for tracking.
`CardSerialNumber`	`string`	Serial number of the card.
`CardNumber`	`string`	Card number.
`IdNumber`	`string`	Emirates ID number.
`Timestamp`	`string`	Timestamp of the response.
`ValidityInterval`	`int`	Validity interval of the response.
`ResponseStatus`	`ResponseStatus`	Status of the response.
`Status`	`string`	Status string.
`XmlString`	`string`	Digitally signed Toolkit Response XML.
`DetachedSignature`	`byte[]`	Detached signature (for AdES operations).

6.1.2. CardPublicData

Extends `ToolkitResponse`. Contains public data read from the Emirates ID Card.

Property	Type	Description
`IdNumber`	`string`	Emirates ID number.
`CardNumber`	`string`	Card number.
`CardHolderPhoto`	`byte[]`	Card holder photograph (JPEG).
`HolderSignatureImage`	`byte[]`	Handwritten signature image.
`NonModifiablePublicData`	`NonModifiablePublicData`	Non-modifiable data attributes.

Property	Type	Description
`ModifiablePublicData`	`ModifiablePublicData`	Modifiable data attributes.
`HomeAddress`	`HomeAddress`	Home address data.
`WorkAddress`	`WorkAddress`	Work address data.

6.1.3. CardCertificates

Extends `ToolkitResponse`. Contains PKI certificates from the Emirates ID Card.

Property	Type	Description
`AuthenticationCertificate`	`byte[]`	Authentication certificate data.
`SigningCertificate`	`byte[]`	Digital signing certificate data.

6.1.4. SignatureResponse

Extends `ToolkitResponse`. Contains the result of a signing operation.

Property	Type	Description
`Signature`	`byte[]`	The digital signature bytes.

6.2. Structures

6.2.1. SigningContext

Properties for digital signature operations (PAdES, XAdES, CAdES).

Property	Type	Description
`SignatureLevel`	`SignatureLevel`	Signature level (Baseline-B, T, LT, or LTA).
`PackagingMode`	`PackagingMode`	Packaging mode (Enveloped, Enveloping, or Detached).
`Location`	`SignerLocation`	Signer's location information.
`TsaUrl`	`string`	Timestamp Authority URL (required for Baseline-T and above).
`OcspUrl`	`string`	OCSP responder URL for certificate revocation checking.
`CertPath`	`string`	Path to certificate store for chain validation.
`EncodedPin`	`string`	ID Card PKI PIN, encrypted and Base64-encoded.

6.2.2. VerificationContext

Properties for signature verification operations.

Property	Type	Description
`OcspUrl`	`string`	OCSP responder URL.
`CertPath`	`string`	Path to certificate store for chain validation.
`ReportType`	`ReportType`	Type of verification report to generate.
`IsDeattached`	`bool`	`true` for detached signature verification.

6.2.3. PadesSignParams

PAdES-specific signature properties for visible signature appearance.

Property	Type	Description
`SignReason`	`string`	Reason for signing.
`SignerLocation`	`string`	Signer's location string.
`SignerContactInfo`	`string`	Signer's contact information.
`SignatureXAxis`	`int`	X-axis position of the visible signature.
`SignatureYAxis`	`int`	Y-axis position of the visible signature.
`SignatureImage`	`string`	Path to signature image file.
`FontName`	`string`	Font name for signature text.
`BackgroundColor`	`string`	Background color of the signature field.
`FontColor`	`string`	Font color for signature text.
`FontSize`	`int`	Font size for signature text.
`SignatureText`	`string`	Text to display in the signature field.
`SignVisible`	`int`	1 for visible signature, 0 for invisible.
`NamePosition`	`SignerNamePosition`	Position of the signer's name relative to the signature.
`PageNumber`	`int`	Page number for the visible signature.

6.2.4. SignerLocation

Signer's location for digital signature metadata.

Property	Type	Description
`CountryCode`	`string`	Country code (e.g., "AE").
`StateOrProvince`	`string`	State or province name.

Property	Type	Description
`PostalCode`	`string`	Postal code.
`Locality`	`string`	City or locality name.
`Street`	`string`	Street address.

6.2.5. MRZData

Parsed Machine Readable Zone data.

Property	Type	Description
`DocumentType`	`string`	Document type identifier.
`IssuedCountry`	`string`	Country of issuance.
`CardNumber`	`string`	Card number.
`IdNumber`	`string`	Emirates ID number.
`DateOfBirth`	`string`	Date of birth.
`Gender`	`string`	Gender.
`CardExpiryDate`	`string`	Card expiry date.
`Nationality`	`string`	Nationality.
`FullName`	`string`	Full name of the card holder.

6.2.6. ToolkitResponseData

Validated Toolkit XML Response attributes.

Property	Type	Description
`Service`	`string`	Toolkit service name.
`Action`	`string`	Action performed.
`RequestId`	`string`	Request ID from the application.
`Nonce`	`string`	Server-generated nonce.
`CorrelationId`	`string`	Correlation identifier.
`CSN`	`string`	Card serial number.
`CardNumber`	`string`	Card number.
`IdNumber`	`string`	Emirates ID number.
`TimeStamp`	`string`	Response timestamp.

Property	Type	Description
ValidityInterval	int	Response validity interval.

6.2.7. ConfigCertExpiryDate

Certificate expiry dates for Toolkit configuration files.

Property	Type	Description
ConfigVGCertExpiry	string	VG configuration certificate expiry date.
ConfigLVCertExpiry	string	Local validation certificate expiry date.
ServerTLSCertExpiry	string	Server TLS certificate expiry date.
ConfigAGCertExpiry	string	Agent configuration certificate expiry date.
LicenseExpiry	string	SDK license expiry date.

6.2.8. DataProtectionKey

VG public key for encoding sensitive parameters.

Property	Type	Description
PublicKey	byte[]	Data protection public key bytes.
Modulus	string	RSA public key modulus.
Exponent	string	RSA public key exponent.

6.2.9. FingerData

Biometric reference data for fingerprint operations.

Property	Type	Description
FingerId	int	Finger reference identifier.
FingerIndex	FingerIndex	Finger index value.

7. Enumerations

7.1. SignatureLevel

Signature level for advanced digital signatures.

Value	Name	Description
1	`BaselineB`	Basic signature with signer's certificate.
2	`BaselineLt`	Long-term validation with revocation data.
3	`BaselineLta`	Long-term archival with archival timestamp.
4	`BaselineT`	Signature with trusted timestamp.

7.2. PackagingMode

Packaging mode for digital signatures.

Value	Name	Description
1	`Enveloped`	Signature is embedded within the signed document.
2	`Enveloping`	Signed document is embedded within the signature.
3	`Detached`	Signature is separate from the signed document.

7.3. ReportType

Verification report detail level.

Value	Name	Description
1	`Detailed`	Comprehensive verification report.
2	`Diagnostic`	Diagnostic-level report.
3	`Simple`	Summary verification result.

7.4. SignerNamePosition

Position of the signer's name in a visible PAdES signature.

Value	Name	Description
1	`Left`	Name positioned to the left.
2	`Right`	Name positioned to the right.
3	`Top`	Name positioned above.
4	`Bottom`	Name positioned below.

7.5. PublicDataEFType

Elementary File types for `ReadPublicDataEF`.

Value	Name	Description
1	`IdnCn`	Identification and Card Number.
2	`RootCertificate`	Root Certificate.
3	`NonModifiableData`	Non-Modifiable Public Data.
4	`ModifiableData`	Modifiable Public Data.
5	`Photography`	Card holder photograph.
6	`SignatureImage`	Handwritten signature image.
7	`HomeAddress`	Home address.
8	`WorkAddress`	Work address.

7.6. FileType

File types for `ReadData` operations.

Value	Name	Description
0	`HealthInsurance`	Health insurance data.
1	`HealthInsuranceProtected`	Protected health insurance data.
2	`ModData`	Modifiable data.
3	`HomeAddress`	Home address.
4	`WorkAddress`	Work address.
5	`HeadOfFamily`	Head of family data.
6–9	`Wife1–Wife4`	Spouse data (up to 4).
10–29	`Child1–Child20`	Child data (up to 20).

7.7. ContainerType

Container types for `UpdateData` operations.

Value	Name	Description
1	`HealthAndInsurance`	Health and insurance container.
2	`ModifiableData`	Modifiable data container.
3	`Address`	Address container.
4	`FamilyBook`	Family book container.

7.8. FingerIndex

Finger index values for biometric operations.

Value	Finger
0	None
3	No specific finger
5	Right Thumb
6	Left Thumb
9	Right Index
10	Left Index
13	Right Middle
14	Left Middle
17	Right Ring
18	Left Ring
21	Right Little
22	Left Little

7.9. InterfaceType

Card communication interface type.

Value	Interface
1	Contact
2	NFC

7.10. ExceptionType

Type of Toolkit exception.

Value	Description
`ToolkitError`	General Toolkit error.
`CardPinError`	PIN-related error with remaining attempt count.

8. Exception Handling

8.1. ToolkitException

All Toolkit API methods throw `ToolkitException` on error. This class extends `System.Exception` and provides additional properties for error diagnosis.

Property	Type	Description
`Code`	`int`	Toolkit error code. Refer to the "ID Card Toolkit References" document for the complete error code list.
`Message`	`string`	Human-readable error description.
`ExceptionType`	`ExceptionType`	`ToolkitError` for general errors; `CardPinError` for PIN-related errors.
`AttemptsLeft`	`int`	Remaining PIN attempts before the card is blocked. Only meaningful when `ExceptionType` is `CardPinError`.
`VGResponse`	`string`	Raw Validation Gateway response XML, if available.

Example: Error Handling

```
try
{
    ToolkitResponse response = cardReader.AuthenticatePki(encodedPin);
}
catch (ToolkitException tEx)
{
    if (tEx.ExceptionType == ExceptionType.CardPinError)
    {
        Console.WriteLine($"Wrong PIN. Attempts left: {tEx.AttemptsLeft}");
    }
    else
    {
        Console.WriteLine($"Error {tEx.Code}: {tEx.Message}");
    }
}
```